



数字信号处理器及其解决方案提供商

FDM320RV335 软件库 使 用手册

版本V1.0

青岛本原微电子有限公司

目 录

1. 概述	- 1 -
1.1. FDM320RV335 软件库介绍	- 1 -
1.2. 软件库目录结构与使用说明	- 1 -
2. 数字信号处理软件库	- 1 -
2.1. 定点数字信号处理软件库	- 2 -
2.1.1. 快速傅里叶变换函数库	- 2 -
2.1.2. 滤波函数库	- 8 -
2.2. 浮点数字信号处理软件库	- 11 -
2.2.1. 快速傅里叶变换函数库	- 11 -
2.2.2. 滤波函数库	- 18 -
2.2.3. 向量函数库	- 20 -
3. 数学软件库	- 34 -
3.1. ATAN_F32	- 35 -
3.2. ATAN2_F32	- 35 -
3.3. COS_F32	- 36 -
3.4. DIV_F32	- 36 -
3.5. EXP_F32	- 37 -
3.6. ISQRT_F32	- 37 -
3.7. LOG_F32	- 38 -
3.8. POW_F32	- 38 -
3.9. SIN_F32	- 39 -
3.10. SINCOS_F32	- 39 -
3.11. SQRT_F32	- 40 -

1. 概述

1.1. FDM320RV335 软件库介绍

FDM320RV335 软件库是为 FDM320RV335 编写的一个高度优化的应用程序函数集合，分为数字信号处理软件库和数学库，其中数字信号处理软件库又分为浮点软件库和定点软件库，充分利用了 FDM320RV335 的浮点处理单元及定点处理单元的潜力。为方便用户使用，软件库为每个函数提供了汇编程序、参考程序以及使用函数的示例程序，所有的函数均在 FDM320RV335 上进行了验证。

1.2. 软件库目录结构与使用说明

FDM320RV335 软件库结构如图 1-1 所示。其中 **performance** 是为函数提供的示例应用程序；**source** 是函数运行的汇编程序。汇编程序、参考程序和参考程序具有相同的结构，其中 **dsp** 是数字信号处理软件库，分为定点软件库和浮点软件库，包含快速傅里叶变换（fft）函数、滤波函数（filter）和向量函数（vector）等数字信号处理领域常用的函数。**math** 是数学软件库，包含三角函数、反三角函数、对数函数和指数函数等常用数学领域函数。

```
> performance
v reference
  v dsp
    v fixedpoint
      > fft
      > filter
    v FPU/fpu32
      > fft
      > filter
      > utility
      > vector
  > math
> source
```

图 1-1 软件库目录结构

2. 数字信号处理软件库

数字信号处理软件库(DSP Library) 是为 FDM320RV335 编写的一组高度优化的数

字信号处理应用函数集合。这些函数使 C/C++ 程序员能够充分利用 FDM320RV335 的性能潜力。本文档提供了库中每个函数的描述。每个库例程都提供了汇编程序、参考程序以及使用函数的示例程序。对于当前版本，所有示例都是针对 FDM320RV335 编写的，并在 FDM320RV335 平台上进行了测试。

数字信号处理软件库包括定点数字信号处理软件库与浮点数字信号处理软件库。其中，定点数字信号处理软件库包括快速傅里叶变换函数库(FFT Library)与滤波函数库(Filter Library)，浮点数字处理软件库包括快速傅里叶变换函数库、滤波函数库和向量函数库(Vector Library)。

2.1. 定点数字信号处理软件库

2.1.1. 快速傅里叶变换函数库

API 汇总:

函数名称	函数接口
FFT32_win	void FFT32_win(CFFT32_Handle);
FFT32_brev	void FFT32_brev(int32_t *src, int32_t *dst, uint16_t size);
FFT32_calc	void FFT32_calc(CFFT32_Handle);
FFT32_init	void FFT32_init(CFFT32_Handle);
FFT32_mag	void FFT32_mag(CFFT32_Handle);
RFFT32_split	void RFFT32_split(RFFT32_Handle hnd);
RFFT32_win	void RFFT32_win(RFFT32_Handle hnd);

2.1.1.1. FFT32_win

- 功能描述:

该函数使用 FFT 模块的 ipcbptr 指向的数据和 winptr 指向的窗口系数（以 Q31 格式表示）对计算缓冲区中的数据进行窗口处理，以调整信号的时域特性，减小频谱泄漏。

- 函数原型:

```
void CFFT32_win(CFFT32_Handle handle);
```

- 函数参数:

CFFT32_Handle 为一个指针，指向一个如下定义的结构体:

```
typedef struct {
    int32_t *ipcbptr; // 指向输入计算 buffer 的指针
    int32_t *tfp_ptr   // 指向旋转因子表的指针
    int16_t size;      // FFT 的大小 (2 的幂, 最大值 4096)
    int16_t nrstage;   // FFT 的计算级数(=log2(size))
    int16_t *magp_ptr; // 指向幅度 buffer 的指针
    int16_t *winp_ptr; // 窗口系数的指针
    int16_t peakmag;   // 峰值频谱幅值
    int16_t peakfrq;   // 峰值幅度的频率
    int16_t ratio;     // ratio=4096/size
    void (*init)(void); // 指向 FFT 初始化函数的指针
    void (*izero)(void *); // 指向虚部清零函数的指针
    void (*calc)(void *); // 指向 FFT 计算函数的指针
    void (*mag)(void *); // 指向 FFT 幅度函数的指针
    void (*win)(void *); // 指向 FFT 窗口函数的指针
}CFFT32;
```

- 函数返回值:

该函数无返回值。计算结果存放在 ipcbptr 所指向的位置。

- 函数性能:

函数名称	数据大小	周期数
CFFT32_win	CFFT SIZE: 512	4674

- 注意点

缓冲区*ipcbptr、*magp_ptr 的大小应为 FFT SIZE 的两倍。

2.1.1.2. FFT32_brev

- 功能描述:

该函数按顺序读取 2N 点真实数据样本，并按位反序顺序将其重新排序写入，以满足复数 FFT 的位反转要求。该函数仅对 src[0], src[2]...进行反序排列。

- 函数原型:

```
void FFT32_brev(int32_t *src, int32_t *dst, uint16_t size)
```

- 函数参数:

src: 指向有序数据样本的指针

dst: 指向目标数组的指针，目标数组缓冲区大小必须为 2N。

size: 要以复数形式进行位反转的实际数据样本数 (N)；它应该是 2 的幂。

- 函数返回值:

该函数没有返回值。计算结果存放在 dst 指向位置。

- 函数性能:

函数名称	数据大小	周期数
FFT32_brev	RFFT SIZE:256	1895

2.1.1.3. FFT32_brev2

- 功能描述:

该函数按顺序读取 2N 点真实数据样本，并按位反序顺序将其重新排序写入，以满足复数 FFT 的位反转要求。该函数同时反序(k,k+1)，(k+2,k+3) ... 两个数据。

- 函数原型:

```
void FFT32_brev(int32_t *src, int32_t *dst, uint16_t size)
```

- 函数参数:

src: 指向有序数据样本的指针

dst: 指向目标数组的指针，目标数组缓冲区大小必须为 2N。

size: 要以复数形式进行位反转的实际数据样本数 (N)；它应该是 2 的幂。

- 函数返回值:

该函数没有返回值。计算结果存放在 dst 指向位置。

- 函数性能:

函数名称	数据大小	周期数
FFT32_brev	RFFT SIZE:256	1895

2.1.1.4. FFT32_calc

- 功能描述:

该模块对 ipcbptr 指向的位反序数据序列（以 Q30 格式表示）上执行 N 点 FFT 计

算，并产生表示频域信息的按顺序数据（以 Q30 格式表示）。输出结果覆盖 ipcbptr 指向的地址。

- 函数原型:

```
void FFT32_calc(CFFT32_Handle handle);
```

- 函数参数:

CFFT32_Handle 的定义见 2.1.1.1 节。

- 函数返回值:

该函数没有返回值。

- 函数性能:

函数名称	数据大小	周期数
FFT32_calc	RFFT SIZE: 256	30846

- 注意点

输入和输出数据都是以 Q30 格式表示的。计算缓冲区的大小为 2N 个长字（FFT 长度的两倍）。

2.1.1.5. FFT32_init

- 功能描述:

FFT 初始化例程用旋转因子表的地址更新旋转因子指针。旋转因子被存放到 tfptr 指向地址。包含最大 3072 个条目（32 位字），以方便多达 4096 个点的复杂 FFT 计算。该表有 3N/4 个条目，其中 N 是支持的最大 FFT。

- 函数原型:

```
void FFT32_init(CFFT32_Handle);
```

- 函数参数:

CFFT32_Handle 的定义见 2.1.1.1 节。

- 函数返回值:

该函数没有返回值。其计算结果用于更新 CFFT32 中 tfptr 指向的旋转因子表。

- 函数性能:

函数名称	数据大小	周期数
FFT32_init	RFFT SIZE: 256	-

2.1.1.6. FFT32_mag

- 功能描述:

该例程获取复数 FFT 输出的幅度平方（以 Q30 格式表示），并将结果（以 Q30 格式表示）根据复数 FFT 模块的 **magptr** 元素的指令，存储在计算缓冲区或专用数组中。

- 函数原型:

```
void FFT32_mag(CFFT32_Handle hnd);
```

- 函数参数:

CFFT32_Handle 的定义见 2.1.1.1 节。

- 函数返回值:

该函数没有返回值。其计算放在 **magptr** 所指向的 **buffer** 中。

- 函数性能:

函数名称	数据大小	周期数
FFT32_mag	RFFT SIZE: 256	2078

- 注意点

请注意，幅度输出以 Q30 格式存储。用于保存幅度输出的数组大小为 N 个长字（等于 FFT 长度）。

2.1.1.7. RFFT32_split

- 功能描述:

Split 函数从 N 点复数 FFT 的输出中获取 N 点实值输入序列的前 $N/2 + 1$ 个复数频谱频段。因此，计算缓冲区的大小为 $N + 2$ ，以复数形式存储 $N/2 + 1$ 个频谱频段。

- 函数原型:

```
void RFFT32_split(RFFT32_Handle hnd);
```

- 函数参数:

RFFT32_Handle 为一个指针，指向一个如下定义的结构体：

```
typedef struct {
    int32_t *ipcbptr; // 指向输入计算 buffer 的指针
    int32_t *tfpstr    // 指向旋转因子表的指针
    int16_t size;      // FFT 的大小 (2 的幂, 最大值 4096)
    int16_t nrstage;   // FFT 的计算级数(=log2(size))
    int16_t *magptr;   // 指向幅度 buffer 的指针
    int16_t *winptr;   // 窗口系数的指针
    int16_t peakmag;   // 峰值频谱幅值
    int16_t peakfrq;   // 峰值幅度的频率
    int16_t ratio;     // 旋转因子表的跳过值
    void (*init)(void); // 指向 FFT 初始化函数的指针
    void (*izero)(void *); // 指向虚部清零函数的指针
    void (*calc)(void *); // 指向 FFT 计算函数的指针
    void (*mag)(void *); // 指向 FFT 幅度函数的指针
    void (*win)(void *); // 指向 FFT 窗口函数的指针
}RFFT32;
```

- 函数返回值:

该函数没有返回值。其计算放在 CFFT32 中 magptr 所指向的 buffer 中。

- 函数性能:

函数名称	数据大小	周期数
RFFT32_split	RFFT SIZE: 256	9027

2.1.1.8. RFFT32_win

- 功能描述:

该函数使用 FFT 模块的 winptr 元素指向的窗口系数（以 Q30 格式表示）对计算缓冲区中的数据进行窗口处理，以调整信号的时域特性，减小频谱泄漏。

- 函数原型:

```
void RFFT32_win(RFFT32_Handle hnd);
```

- 函数参数:

RFFT32_Handle 的定义见 2.1.1.6 节。

- 函数返回值:

该函数没有返回值。其计算放在 RFFT32 中 ipcbptr 所指向的 buffer 中。

● 函数性能:

函数名称	数据大小	周期数
RFFT32_win	RFFT SIZE: 256	5172

● 注意点

一旦输入数据以位反转格式重新排序，就应该调用窗口函数。

2.1.2. 滤波函数库

API 汇总:

函数名称	函数接口
FIR16_calc	void FIR16_calc(FIR16_handle hndFIR16);
FIR32_calc	void FIR32_calc(FIR32_handle hndFIR32);
IIR16_calc	void IIR16_calc(IIR16_handle hndIIR16);
IIR32_calc	void IIR32_calc(IIR32_handle hndIIR32);

2.1.2.1. FIR16_calc

● 功能描述:

该模块实现了 16bit 的有限脉冲响应（FIR）滤波器。

● 函数原型:

```
void FIR16_calc(FIR16_handle hndFIR16);
```

● 函数参数:

FIR16_handle 是一个指向如下定义的结构体的指针:

```
typedef struct {
    int32_t *coeff_ptr; // 指向滤波器系数的指针
    int32_t *dbuffer_ptr; // 延迟缓冲区指针
    int16_t cindex; // 循环缓冲区索引
    int16_t order; // 滤波器的阶数
    fsize_t input; // 最新的输入样本。fsize_t 为 int16_t。
    fsize_t output; // 滤波器输出
    void (*init)(void *); // 指向初始化函数的指针
    void (*calc)(void *); // 指向计算函数的指针
}FIR16;
```

● 函数返回值:

该函数没有返回值。其计算放在 FIR16 的 output 中。

● 函数性能:

函数名称	数据大小	周期数
FIR16_calc	FIR Order: 32 Signal Length: 128	133

2.1.2.2. FIR32_calc

● 功能描述:

该模块实现了 32bit 的有限脉冲响应（FIR）滤波器。

● 函数原型:

```
void FIR32_calc(FIR32_handle hndFIR32);
```

● 函数参数:

FIR32_handle 是一个指向如下定义的结构体的指针:

```
typedef struct {
    int32_t *coeff_ptr; // 指向滤波器系数的指针
    int32_t *dbuffer_ptr; // 延迟缓冲区指针
    int16_t cbindex; // 循环缓冲区索引
    int16_t order; // 滤波器的阶数
    fsize_t input; // 最新的输入样本。fsize_t 为 int32_t。
    fsize_t output; // 滤波器输出
    void (*init)(void *); // 指向初始化函数的指针
    void (*calc)(void *); // 指向计算函数的指针
}FIR32;
```

● 函数返回值:

该函数没有返回值。其计算放在 FIR32 的 output 中。

● 函数性能:

函数名称	数据大小	周期数
FIR32_calc	FIR Order: 43 Signal Length: 128	166

2.1.2.3. IIR16_calc

- 功能描述:

该模块实现了 16bit 的无限脉冲响应 (Infinite Impulse Response)。

- 函数原型:

```
void IIR16_calc(IIR16_handle hndIIR16);
```

- 函数参数:

IIR16_handle 是一个指向如下定义的结构体的指针:

```
typedef struct {
    void (*init)(void *);    // 指向初始化函数的指针
    void (*calc)(void *);    // 指向计算函数的指针
    int16_t *coeff_ptr;      // 滤波器系数的指针
    int16_t *dbuffer_ptr;    // 延迟缓冲区指针
    uint16_t nbiqu;          // 双二阶滤波器的数量
    int16_t input;           // 最新的输入样本。
    int16_t isf;             // 输入比例因子
    uint16_t qformat;        // 系数的 Q 格式
    int16_t output;          // 滤波器输出
} IIR5BIQ16;
```

- 函数返回值:

该函数没有返回值。其计算放在 IIR5BIQ16 的 output 中。

- 函数性能:

函数名称	数据大小	周期数
IIR16_calc	Signal Length: 100 Number of Biquads: 4	150

2.1.2.4. IIR32_calc

- 功能描述:

该模块实现了 32bit 的无限脉冲响应 (Infinite Impulse Response)。

- 函数原型:

```
void IIR32_calc(IIR32_handle hndIIR32);
```

● 函数参数:

IIR32_handle 是一个指向如下定义的结构体的指针:

```
typedef struct {
    void (*init)(void *);    // 指向初始化函数的指针
    void (*calc)(void *);    // 指向计算函数的指针
    fsize_t *coeff_ptr;      // 滤波器系数的指针。fsize_t 为 int32_t。
    fsize_t *dbuffer_ptr;    // 延迟缓冲区指针
    uint16_t nbiqu;          // 双二阶滤波器的数量
    fsize_t input;           // 最新的输入样本。
    fsize_t isf;             // 输入比例因子
    fsize_t output32;        // 滤波器输出 (Q30)
    int16_t output16;        // 滤波器输出 (Q15)
    uint16_t qformat;        // 系数的 Q 格式
} IIR5BIQ32;
```

● 函数返回值:

该函数没有返回值。其计算放在 IIR5BIQ32 的 output32 中。

● 函数性能:

函数名称	数据大小	周期数
IIR32_calc	Signal Length: 100 Number of Biquads: 4	150

2.2. 浮点数字信号处理软件库

2.2.1. 快速傅里叶变换函数库

API 汇总:

函数名称	函数接口
CFFT_f32_brev	void CFFT_f32_brev(CFFT_F32_STRUCT_Handle hnd);
CFFT_f32_mag	void CFFT_f32_mag(CFFT_F32_STRUCT_Handle hnd);
CFFT_f32_phase	void CFFT_f32_phase(CFFT_F32_STRUCT_Handle hnd);
CFFT_f32_sincostable	void CFFT_f32_sincostable(CFFT_F32_STRUCT_Handle hnd);
CFFT_f32_unpack	void CFFT_f32_unpack(CFFT_F32_STRUCT_Handle hnd);
CFFT_f32	void CFFT_f32(CFFT_F32_STRUCT_Handle hnd);
ICFFT_f32	void ICFFT_f32(CFFT_F32_STRUCT_Handle hnd);
RFFT_f32_mag	void RFFT_f32_mag(CFFT_F32_STRUCT_Handle hnd);
RFFT_f32_phase	void RFFT_f32_phase(CFFT_F32_STRUCT_Handle hnd);
RFFT_f32_sincostable	void RFFT_f32_sincostable(CFFT_F32_STRUCT_Handle hnd);

RFFT_f32_win	void RFFT_f32_win(CFFT_F32_STRUCT_Handle hnd);
RFFT_f32	void RFFT_f32(CFFT_F32_STRUCT_Handle hnd);

2.2.1.1. CFFT_f32_brev

- 功能描述:

这个例程将按位反序重新排列一个 N 点复数数据集。

- 函数原型:

```
void CFFT_f32_brev(CFFT_F32_STRUCT_Handle hnd);
```

- 函数参数:

CFFT_F32_STRUCT_Handle 为一个指针, 指向一个如下定义的结构体:

```
typedef struct {
    float *InPtr; // 输入 buffer 的指针
    float *OutPtr; // 输出 buffer 的指针
    float *CoefPtr; // 旋转因子的指针
    float *CurrentInPtr; // FFT 每一级中的输入 buffer 指针
    float *CurrentOutPtr; // FFT 每一级中的输出 buffer 指针
    uint16_t Stages; // FFT 的级数
    uint16_t FFTSize; // FFT 的大小 (复数数据点的数量)
}CFFT_F32_STRUCT;
```

- 函数返回值:

该函数无返回值。计算结果存放在 OutPtr 所指向的位置。

- 函数性能:

函数名称	数据大小	周期数
CFFT_f32_brev	Test Size: 128	2661

2.2.1.2. CFFT_f32_mag

- 功能描述:

该例程计算复数 FFT 的幅度。

- 函数原型:

```
void CFFT_f32_mag(CFFT_F32_STRUCT_Handle hnd);
```

- 函数参数:

CFFT_F32_STRUCT_Handle 的定义见 2.2.1.1 节。

- 函数返回值:

该函数无返回值。计算结果存放在 OutPtr 所指向的位置。

- 函数性能:

函数名称	数据大小	周期数
CFFT_f32_mag	Test Size: 128	2141

2.2.1.3. CFFT_f32_phase

- 功能描述:

复数 FFT 相位。

- 函数原型:

```
void CFFT_f32_phase(CFFT_F32_STRUCT_Handle hnd);
```

- 函数参数:

CFFT_F32_STRUCT_Handle 的定义见 2.2.1.1 节。

- 函数返回值:

该函数无返回值。计算结果存放在 OutPtr 所指向的位置。

- 函数性能:

函数名称	数据大小	周期数
CFFT_f32_phase	Test Size: 128	10152

2.2.1.4. CFFT_f32_sincostable

- 功能描述:

为复数 FFT 生成旋转因子。

- 函数原型:

```
void CFFT_f32_sincostable(CFFT_F32_STRUCT_Handle hnd);
```

- 函数参数:

CFFT_F32_STRUCT_Handle 的定义见 2.2.1.1 节。

- 函数返回值:

该函数无返回值。计算结果存放在 CoefPtr 所指向的位置。

- 函数性能:

函数名称	数据大小	周期数
CFFT_f32_sincostable	Test Size: 128	267656

2.2.1.5. CFFT_f32_unpack

- 功能描述:

将 N 点复数 FFT 的输出解包以获得 2N 点实序列的 FFT。

- 函数原型:

```
void CFFT_f32_unpack(CFFT_F32_STRUCT_Handle hnd);
```

- 函数参数:

CFFT_F32_STRUCT_Handle 的定义见 2.2.1.1 节。

- 函数返回值:

该函数无返回值。计算结果存放在 OutPtr 所指向的位置。

- 函数性能:

函数名称	数据大小	周期数
CFFT_f32_unpack	Test Size: 256	4431

2.2.1.6. CFFT_f32

- 功能描述:

复数快速傅立叶变换。此例程计算 N 个点 ($N = 2^n$, $n = 5 : 10$) 复数输入的 32 位浮点 FFT。

- 函数原型:

```
void CFFT_f32(CFFT_F32_STRUCT_Handle hnd);
```

- 函数参数:

CFFT_F32_STRUCT_Handle 的定义见 2.2.1.1 节。

- 函数返回值:

该函数无返回值。计算结果存放在 OutPtr 所指向的位置。

- 函数性能:

函数名称	数据大小	周期数
CFFT_f32	Test Size: 128	17577

2.2.1.7. ICFFT_f32

- 功能描述:

逆复数 FFT。该例程计算 N 个点 ($N = 2^n$, $n = 5 : 10$) 复数输入的 32 位浮点逆 FFT。它使用正向 FFT 来实现, 首先交换输入的实部和虚部, 运行正向 FFT, 然后交换输出的实部和虚部以获得最终结果。

- 函数原型:

```
void ICFFT_f32(CFFT_F32_STRUCT_Handle hnd);
```

- 函数参数:

CFFT_F32_STRUCT_Handle 的定义见 2.2.1.1 节。

- 函数返回值:

该函数无返回值。计算结果存放在 OutPtr 所指向的位置。

- 函数性能:

函数名称	数据大小	周期数
ICFFT_f32	Test Size: 1024	64436

2.2.1.8. RFFT_f32_mag

- 功能描述:

这个函数计算实数 FFT 的幅度。

- 函数原型:

```
void RFFT_f32_mag(CFFT_F32_STRUCT_Handle hnd);
```

- 函数参数:

CFFT_F32_STRUCT_Handle 的定义见 2.2.1.1 节。

- 函数返回值:

该函数无返回值。计算结果存放在 OutPtr 所指向的位置。

- 函数性能:

函数名称	数据大小	周期数
RFFT_f32_mag	Test Size: 256	4218

2.2.1.9. RFFT_f32_phase

- 功能描述:

实数 FFT 相位。

- 函数原型:

```
void RFFT_f32_phase(CFFT_F32_STRUCT_Handle hnd);
```

- 函数参数:

CFFT_F32_STRUCT_Handle 的定义见 2.2.1.1 节。

- 函数返回值:

该函数无返回值。计算结果存放在 OutPtr 所指向的位置。

- 函数性能:

函数名称	数据大小	周期数
RFFT_f32_phase	Test Size: 256	21467

2.2.1.10. RFFT_f32_sincostable

- 功能描述:

生成实数 FFT 的旋转因子。

- 函数原型:

```
void RFFT_f32_sincostable(CFFT_F32_STRUCT_Handle hnd);
```

- 函数参数:

CFFT_F32_STRUCT_Handle 的定义见 2.2.1.1 节。

- 函数返回值:

该函数无返回值。计算结果存放在 CoefPtr 所指向的位置。

- 函数性能:

函数名称	数据大小	周期数
RFFT_f32_sincostable	Test Size: 256	267656

2.2.1.11. RFFT_f32_win

- 功能描述:

用于 32 位实数 FFT 的窗口函数。

- 函数原型:

```
void RFFT_f32_win(CFFT_F32_STRUCT_Handle hnd);
```

- 函数参数:

CFFT_F32_STRUCT_Handle 的定义见 2.2.1.1 节。

- 函数返回值:

该函数无返回值。计算结果存放在 InPtr 所指向的位置。

- 函数性能:

函数名称	数据大小	周期数
RFFT_f32_win	Test Size: 1024	1197

2.2.1.12. RFFT_f32

- 功能描述:

实数快速傅里叶变换（RFFT）。该例程计算 N 个点（ $N = 2^n$, $n = 5 : 10$ ）实数输入的 32 位浮点 FFT。该函数在阶段 1、2 和 3 的计算中按位逆序重新排列输入。该例程以乒乓方式使用两个缓冲区，即在每个 FFT 阶段之后，输出和输入缓冲区分别成为下一个阶段的输入和输出缓冲区。

- 函数原型:

```
void RFFT_f32(CFFT_F32_STRUCT_Handle hnd);
```

- **函数参数:**

CFFT_F32_STRUCT_Handle 的定义见 2.2.1.1 节。

- **函数返回值:**

该函数无返回值。计算结果存放在 OutPtr 所指向的位置。

- **函数性能:**

函数名称	数据大小	周期数
RFFT_f32	Test Size: 256	18509

2.2.2. 滤波函数库

API 汇总:

函数名称	函数接口
FIR_f32_calc	void FIR_f32_calc(FIR_f32_Handle hndFIR_f32);
IIR_f32_calc	void IIR_f32_calc(IIR_f32_Handle hndIIR_f32);

2.2.2.1. FIR_f32_calc

- **功能描述:**

该模块实现了有限脉冲响应（FIR）滤波器。

- **函数原型:**

```
void FIR_f32_calc(FIR_f32_Handle hndFIR_f32);
```

- **函数参数:**

FIR_f32_Handle 为一个指针，指向一个如下定义的结构体:

```
typedef struct {
    float *coeff_ptr; // 指向滤波器系数的指针。
    float *dbuffer_ptr; // 延迟缓冲区指针
    int cbindex; // 循环缓冲区索引
    int order; // 滤波器的阶数
    float input; // 最新的输入样本
    float output; // 滤波器输出
    void (*init)(void *); // 指向初始化函数的指针
    void (*calc)(void *); // 指向计算函数的指针
    float *in_ptr; // 可以将其初始化为 FIRstate
```

```
}FIR_f32;
```

- **函数返回值:**

该函数没有返回值。其计算放在 FIR_f32 的 output 中。

- **函数性能:**

函数名称	数据大小	周期数
FIR_f32_calc	Test Size: 256 order: 64	614

2.2.2.2. IIR_f32_calc

- **功能描述:**

该模块实现了无限脉冲响应 (Infinite Impulse Response)。

- **函数原型:**

```
void IIR_f32_calc(IIR_f32_Handle hndIIR_f32);
```

- **函数参数:**

IIR_f32_Handle 为一个指针, 指向一个如下定义的结构体:

```
typedef struct {
    float *p_coeff_A; // 指向分母系数的指针。
    float *p_coeff_B; // 指向分子系数的指针。
    float *p_dbuffer; // 延迟缓冲区指针
    float *p_input;   // 最新的输入样本的指针
    float *p_output;  // 滤波器输出的指针
    float *p_scale;   // 指向双二次(biquad)比例值的指针。
    uint16_t order;   // 滤波器的阶数
    void (*init)(void *); // 指向初始化函数的指针
    void (*calc)(void *); // 指向计算函数的指针
}IIR_f32;
```

- **函数返回值:**

该函数没有返回值。其计算放在 IIR_f32 的 output 中。

- **函数性能:**

函数名称	数据大小	周期数
IIR_f32_calc	Test Size: 256 Filter Order: 6	199

2.2.3. 向量函数库

API 汇总:

函数名称	函数接口
abs_SP_CV	void abs_SP_CV(float32 *y, const complex_float *x, const Uint16 N);
add_SP_CSxCV	void add_SP_CSxCV(complex_float *y, const complex_float *x, const complex_float c, const Uint16 N);
add_SP_CVxCV	void add_SP_CVxCV(complex_float *y, const complex_float *w, const complex_float *x, const Uint16 N);
iabs_SP_CV	void iabs_SP_CV(float32 *y, const complex_float *x, const Uint16 N);
mac_SP_CVxCV	complex_float mac_SP_CVxCV(const complex_float *w, const complex_float *x, const uint16_t N);
mac_SP_RVxCV	complex_float mac_SP_RVxCV(const complex_float *w, const float32 *x, const Uint16 N);
maxidx_SP_RV	Uint16 maxidx_SP_RV(const float32 *x, const Uint16 N);
mean_SP_CV_2	complex_float mean_SP_CV_2(const complex_float *x, const Uint16 N);
median_SP_RV	float median_SP_RV(float *, uint16_t);
mpy_SP_CSxCS	complex_float mpy_SP_CSxCS(const complex_float w, const complex_float x);
mpy_SP_CVxCV	void mpy_SP_CVxCV(complex_float *y, const complex_float *w, const complex_float *x, const Uint16 N);
mpy_SP_CVxCVC	void mpy_SP_CVxCVC(complex_float *y, const complex_float *w, const complex_float *x, const Uint16 N);
mpy_SP_RMxRM	void mpy_SP_RMxRM(float *y, const float *w, const float *x, const uint16_t m, const uint16_t n, const uint16_t p);
mpy_SP_RSxRV	void mpy_SP_RSxRV(float32 *y, const float32 *x, const float32 c, const Uint16 N);
mpy_SP_RSxRVxRV	void mpy_SP_RSxRVxRV(float32 *y, const float32 *w, const float32 *x, const float32 c, const Uint16 N);
mpy_SP_RVxCV	void mpy_SP_RVxCV(complex_float *y, const complex_float *w, const float32 *x, const Uint16 N);
mpy_SP_RVxRV	void mpy_SP_RVxRV(float32 *y, const float32 *w, const float32 *x, const Uint16 N);
qsort_SP_RV	void qsort_SP_RV(void *base, const uint16_t nmem);
rnd_SP_RS	float32 rnd_SP_RS(const float32 x);
sub_SP_CSxCV	void sub_SP_CSxCV(complex_float *y, const complex_float *x, const complex_float c, const Uint16 N);
sub_SP_CVxCV	void sub_SP_CVxCV(complex_float *y, const complex_float *w,

```
const complex_float *x, const Uint16 N);
```

2.2.3.1. abs_SP_CV

- 功能描述:

计算复向量的绝对值

$$y[i] = \text{sqrtf}(x_{re}[i]^2 + x_{im}[i]^2)$$

- 函数原型:

```
void abs_SP_CV(float32 *y, const complex_float *x, const Uint16 N);
```

- 函数参数:

y: 输出数组;

x: 输入数组, complex_float 的定义如下:

```
typedef struct {
    float32 dat[2]; // 元素 dat[0]是实部, dat[1]是虚部。
} complex_float;
```

N: x 和 y 数组的长度。

- 函数返回值:

该函数没有返回值。其计算放在数组 y 中。

- 函数性能:

函数名称	数据大小	周期数
abs_SP_CV	vectore size: [99,99,SIZE=98]	1913

2.2.3.2. add_SP_CSxCV

- 功能描述:

复向量的复标量加法:

$$y_{re}[i] = x_{re}[i] + c_{re}$$

$$y_{im}[i] = x_{im}[i] + c_{im}$$

- 函数原型:


```
void add_SP_CSxCV(complex_float *y, const complex_float *x,
                  const complex_float c, const Uint16 N);
```

- 函数参数:

y: 输出复数数组, `complex_float` 的定义见 2.2.3.1 节;

x: 输入复数数组;

c: 输入复数标量;

N: x 与 y 数组的长度。

- 函数返回值:

该函数没有返回值。其计算放在数组 y 中。

- 函数性能:

函数名称	数据大小	周期数
add_SP_CSxCV	vectore size: [64,64,1,SIZE=64]	797

2.2.3.3. add_SP_CVxCV

- 功能描述:

复向量与复向量的加法:

$$y_{re}[i] = w_{re}[i] + x_{re}[i]$$

$$y_{im}[i] = w_{im}[i] + x_{im}[i]$$

- 函数原型:

```
void add_SP_CVxCV(complex_float *y, const complex_float *w,
                  const complex_float *x, const Uint16 N);
```

- 函数参数:

y: 输出复数数组, `complex_float` 的定义见 2.2.3.1 节;

w: 输入复数数组 1;

x: 输入复数数组 2;

N: x, w, y 数组的长度。

- 函数返回值:

该函数没有返回值。其计算放在数组 y 中。

- 函数性能:

函数名称	数据大小	周期数
add_SP_CVxCV	vectore size: [64,64,64,SIZE=64]	923

2.2.3.4. iabs_SP_CV

- 功能描述:

复向量的逆绝对值

$$y[i] = 1 / (\sqrt{(x[i].r * x[i].r) + (x[i].i * x[i].i)})$$

- 函数原型:

```
void iabs_SP_CV(float32 *y, const complex_float *x, const Uint16 N);
```

- 函数参数:

y: 输出数组;

x: 输入数组, complex_float 的定义见 2.2.3.1 节;

N: x 和 y 数组的长度。

- 函数返回值:

该函数没有返回值。其计算放在数组 y 中。

- 函数性能:

函数名称	数据大小	周期数
iabs_SP_CV	vectore size: [99,99,SIZE=98]	3283

2.2.3.5. mac_SP_CVxCV

- 功能描述:

复向量与复向量的乘累加

$$y_{re} = \text{sum}(w_{re}[i] * x_{re}[i] - w_{im}[i] * x_{im}[i])$$

$$y_{im} = \text{sum}(w_{re}[i] * x_{im}[i] + w_{im}[i] * x_{re}[i])$$

- 函数原型:

```
complex_float mac_SP_CVxCV(const complex_float *w,
```

```
const complex_float *x, const uint16_t N);
```

- 函数参数:

w: 第一个输入复数数组, complex_float 的定义见 2.2.3.1 节;

x: 第二个输入复数数组;

N: w 和 x 的长度;

- 函数返回值:

计算结果

- 函数性能:

函数名称	数据大小	周期数
mac_SP_CVxCV	vectore size: [256,128,SIZE=64]	943

2.2.3.6. mac_SP_RVxCV

- 功能描述:

实向量与复向量的乘累加

$$y_{re} = \text{sum}(x[i] * w_{re}[i])$$

$$y_{im} = \text{sum}(x[i] * w_{im}[i])$$

- 函数原型:

```
complex_float mac_SP_RVxCV(const complex_float *w,
                           const float32 *x, const Uint16 N);
```

- 函数参数:

w: 输入复数数组, complex_float 的定义见 2.2.3.1 节;

x: 输入实数数组;

N: w 与 x 的长度。

- 函数返回值:

计算结果。

- 函数性能:

函数名称	数据大小	周期数
mac_SP_RVxCV	vectore size: [64,64,SIZE=64]	554

2.2.3.7. maxidx_SP_RV

- 功能描述:

实数数组最大值的索引

```
if(x[i] > max){
    max = x[i];
    idx = i;
}
```

- 函数原型:

```
Uint16 maxidx_SP_RV(const float32 *x, const Uint16 N);
```

- 函数参数:

x: 输入数组;

N: x 的长度。

- 函数返回值:

x 中最大值的索引。

- 函数性能:

函数名称	数据大小	周期数
maxidx_SP_RV	vectore size: [100,SIZE=100]	910

2.2.3.8. mean_SP_CV_2

- 功能描述:

复向量实部和虚部的平均值。

$$y_{re} = \text{sum}(x_{re}[i])/N$$

$$y_{im} = \text{sum}(x_{im}[i])/N$$

- 函数原型:

```
complex_float mean_SP_CV_2(const complex_float *x, const Uint16 N);
```

- 函数参数:

x: 输入复数数组, complex_float 的定义见 2.2.3.1 节;

N: x 的长度。

- 函数返回值:

计算结果

- 函数性能:

函数名称	数据大小	周期数
mean_SP_CV_2	vectore size: [100,SIZE=100]	755

2.2.3.9. median_SP_RV

- 功能描述:

实际浮点数组的中值。未保留输入数组。此例程使用基于《Numerical recipes in C》第二版中所述算法的快速选择方法，剑桥大学出版社，1992 年，第 8.5 节，ISBN 0-521-43108-5

- 函数原型:

```
float median_SP_RV(float *, uint16_t);
```

- 函数参数:

x: 输入数组;

N: x 的长度;

- 函数返回值:

计算结果

- 函数性能:

函数名称	数据大小	周期数
median_SP_RV	vectore size: [100,SIZE=100]	19753

2.2.3.10. mpy_SP_CSxCS

- 功能描述:

两个浮点数的复数乘法

$$y.r = w.r * x.r - w.i * x.i;$$

$$y.i = w.r * x.i + w.i * x.r;$$

- 函数原型:

```
complex_float mpy_SP_CSxCS(const complex_float w,
                           const complex_float x);
```

- 函数参数:

w: 第一个复数输入;

x: 第二个复数输入;

- 函数返回值:

计算结果

- 函数性能:

函数名称	数据大小	周期数
mpy_SP_CSxCS	vectore size: [1,1]	16

2.2.3.11. mpy_SP_CVxCV

- 功能描述:

复向量与复数的乘积

$$y_{re}[i] = w_{re}[i] * x_{re}[i] - w_{im}[i] * x_{im}[i]$$

$$y_{im}[i] = w_{re}[i] * x_{im}[i] + w_{im}[i] * x_{re}[i]$$

- 函数原型:

```
void mpy_SP_CVxCV(complex_float *y, const complex_float *w,
                  const complex_float *x, const Uint16 N);
```

- 函数参数:

y: 计算结果数组, complex_float 的定义见 2.2.3.1 节;

w: 第一个输入数组;

x: 第二个输入数组;

N: w, x, y 数组的长度。

- 函数返回值:

该函数没有返回值。其计算放在数组 y 中。

- 函数性能:

函数名称	数据大小	周期数
mpy_SP_CVxCV	vectore size: [64,64,SIZE=64]	1507

2.2.3.12. mpy_SP_CVxCVC

- 功能描述:

复向量与复向量的复共轭的乘积

$$xconj_{re}[i] = x_{re}[i]$$

$$xconj_{im}[i] = -x_{im}[i]$$

$$y_{re}[i] = w_{re}[i] * x_{re}[i] - w_{im}[i] * xconj_{im}[i]$$

$$y_{im}[i] = w_{re}[i] * xconj_{im}[i] + w_{im}[i] * x_{re}[i]$$

- 函数原型:

```
void mpy_SP_CVxCVC(complex_float *y, const complex_float *w,
                   const complex_float *x, const Uint16 N);
```

- 函数参数:

y: 结果数组, complex_float 的定义见 2.2.3.1 节;

w: 第一个输入数组;

x: 第二个输入数组;

N: y, w, x 数组的长度。

- 函数返回值:

该函数没有返回值。其计算放在数组 y 中。

- 函数性能:

函数名称	数据大小	周期数
mpy_SP_CVxCVC	vectore size: [64,64,SIZE=64]	1444

2.2.3.13. mpy_SP_RMxRM

- 功能描述:

两个实矩阵的乘积 $y[] = w[] * x[]$, 其中:

w[]的维度为 $m \times n$;

x[]的维度为 $n \times p$;

y[]的维度为 $m \times p$;

```
for(i = 0U; i < m; i++){
    for(j = 0U; j < p; j++){
        *(y + i + j*p) = (fsize_t)0.0;
        for(k = 0U; k < n; k++){
            *(y + i + j*p) += *(w + i + k*m) * *(x + j*n + k);
        }
    }
}
```

- 函数原型:

```
void mpy_SP_RMxRM(float *y, const float *w, const float *x,
                  const uint16_t m, const uint16_t n, const uint16_t p);
```

- 函数参数:

y: 计算结果矩阵;

w: 第一个输入矩阵;

x: 第二个输入矩阵;

m: 矩阵的维度 m;

n: 矩阵的维度 n;

p: 矩阵的维度 p;

- 函数返回值:

该函数没有返回值。其计算放在数组 y 中

- 函数性能:

函数名称	数据大小	周期数
mpy_SP_RMxRM	vectore size: [256,256,256,SIZE=16,SIZE=4,SIZE=16]	14965

2.2.3.14. mpy_SP_RSxRV

- 功能描述:

实标量与实向量的乘积

$$y[i] = c * x[i]$$

- 函数原型:

```
void mpy_SP_RSxRV(float32 *y, const float32 *x, const float32 c,
                  const Uint16 N);
```

- 函数参数:

y: 计算结果数组;

x: 输入数组;

c: 输入标量;

N: x 和 y 的长度。

- 函数返回值:

该函数没有返回值。其计算放在数组 y 中。

- 函数性能:

函数名称	数据大小	周期数
mpy_SP_RSxRV	vectore size: [64,64,1,SIZE=64]	253

2.2.3.15. mpy_SP_RSxRVxRV

- 功能描述:

实数标量、实数向量和实数向量的乘法运算:

$$y[i] = c * w[i] * x[i]$$

- 函数原型:

```
void mpy_SP_RSxRVxRV(float32 *y, const float32 *w,
                     const float32 *x, const float32 c, const Uint16 N);
```

- 函数参数:

y: 计算结果数组;

w: 第一个输入数组;

x: 第二个输入数组;

c: 输入标量;

N: w, x, y 数组的长度;

- 函数返回值:

该函数没有返回值。其计算放在数组 y 中。

- 函数性能:

函数名称	数据大小	周期数
mpy_SP_RSxRVxRV	vectore size: [64,64,64,1,SIZE=64]	512

2.2.3.16. mpy_SP_RVxCV

- 功能描述:

实数向量和复数向量的乘法运算。

$$y_{re}[i] = x[i] * w_{re}[i]$$

$$y_{im}[i] = x[i] * w_{im}[i]$$

- 函数原型:

```
void mpy_SP_RVxCV(complex_float *y, const complex_float *w,
                  const float32 *x, const Uint16 N);
```

- 函数参数:

y: 计算结果复数数组, complex_float 的定义见 2.2.3.1 节;

w: 输入复数数组;

x: 输入实数;

N: w, x, y 数组的长度。

- 函数返回值:

该函数没有返回值。其计算放在数组 y 中。

- 函数性能:

函数名称	数据大小	周期数
mpy_SP_RVxCV	vectore size: [64,64,64,SIZE=64]	669

2.2.3.17. mpy_SP_RVxRV

- 功能描述:

实数向量和实数向量的乘法运算。

$$y[i] = w[i] * x[i]$$

- 函数原型:

```
void mpy_SP_RVxRV(float32 *y, const float32 *w, const float32 *x,
                  const Uint16 N);
```

- 函数参数:

y: 计算结果实数数组;

w: 第一个输入实数数组;

x: 第二个输入实数数组;

N: w, x, y 数组的长度。

- 函数返回值:

该函数没有返回值。其计算放在数组 y 中。

- 函数性能:

函数名称	数据大小	周期数
mpy_SP_RVxRV	vectore size: [64,64,64,SIZE=64]	385

2.2.3.18. qsort_SP_RV

- 功能描述:

对一个实数数组进行排序。

- 函数原型:

```
void qsort_SP_RV(void *base, const uint16_t nmem);
```

- 函数参数:

base: 输入数组的指针;

nmem: 输入数组的长度。

- 函数返回值:

该函数没有返回值。排序后的数组放在输入数组位置。

- 函数性能:

函数名称	数据大小	周期数
qsort_SP_RV	vectore size: [100,SIZE=100]	33252

2.2.3.19. rnd_SP_RS

● 功能描述:

浮点标量的无偏舍入。

● 函数原型:

```
float32 rnd_SP_RS(const float32 x);
```

● 函数参数:

x: 输入浮点值。

● 函数返回值:

舍入后的结果。

● 函数性能:

函数名称	数据大小	周期数
rnd_SP_RS	Input 1 Number	93

2.2.3.20. sub_SP_CSxCV

● 功能描述:

从复数向量中减去复数标量的运算

$$y_{re}[i] = x_{re}[i] - c_{re}$$

$$y_{im}[i] = x_{im}[i] - c_{im}$$

● 函数原型:

```
void sub_SP_CSxCV(complex_float *y, const complex_float *x,
                  const complex_float c, const Uint16 N);
```

● 函数参数:

y: 计算结果复数数组, complex_float 的定义见 2.2.3.1 节;

x: 输入复数数组;

c: 输入复数标量;

N: x 与 y 数组的长度。

● 函数返回值:

该函数没有返回值。其计算放在数组 y 中。

● 函数性能:

函数名称	数据大小	周期数
sub_SP_CSxCV	vectore size: [64,64,1,SIZE=64]	544

2.2.3.21. sub_SP_CVxCV

● 功能描述:

复数向量和复数向量的减法运算

$$y_{re}[i] = w_{re}[i] - x_{re}[i]$$

$$y_{im}[i] = w_{im}[i] - x_{im}[i]$$

● 函数原型:

```
void sub_SP_CVxCV(complex_float *y, const complex_float *w,
                  const complex_float *x, const Uint16 N);
```

● 函数参数:

y: 计算结果复数数组, complex_float 的定义见 2.2.3.1 节;

w: 第一个数组复数数组;

x: 第二个输入复数数组;

N: w, x, y 数组的长度。

● 函数返回值:

该函数没有返回值。其计算放在数组 y 中。

● 函数性能:

函数名称	数据大小	周期数
sub_SP_CVxCV	vectore size: [64,64,64,SIZE=64]	737

3. 数学软件库

API 汇总:

函数名称	函数接口
atan_f32	fsize_t atan_f32(const fsize_t x);
atan2_f32	fsize_t atan2_f32(const fsize_t y,const fsize_t x);

cos_f32	fsize_t cos_f32(const fsize_t theta);
div_f32	fsize_t div_f32(const fsize_t num, const fsize_t den);
exp_f32	fsize_t exp_f32(const fsize_t x);
isqrt_f32	fsize_t isqrt_f32(const fsize_t x);
log_f32	fsize_t log_f32(const fsize_t x);
pow_f32	fsize_t pow_f32(const fsize_t x, const fsize_t y);
sin_f32	fsize_t sin_f32(const fsize_t theta);
sincos_f32	sincos_t sincos_f32(const fsize_t theta);
sqrt_f32	fsize_t sqrt_f32(const fsize_t x);

3.1. atanf

- 功能描述:

单精度浮点数下的反正切函数（以弧度为单位）

- 函数原型:

```
fsize_t atanf(const fsize_t x);
```

- 函数参数:

x: 单精度浮点输入。

- 函数返回值:

浮点数参数 X 的反正切值。

- 函数性能:

函数名称	数据大小	周期数
atanf	Test Size: 256	92

3.2. atan2f

- 功能描述:

单精度浮点数下的双参数反正切函数（以弧度为单位）。

- 函数原型:

```
fsize_t atan2f(const fsize_t y, const fsize_t x);
```

- 函数参数:

y: 第一个单精度浮点参数;

x: 第二个单精度浮点参数;

- 函数返回值:

浮点数参数 X/Y 的四象限反正切值。

- 函数性能:

函数名称	数据大小	周期数
atan2f	Test Size: 256	94

3.3. cosf

- 功能描述:

单精度浮点余弦函数(以弧度为单位)。

- 函数原型:

```
fsize_t cosf(const fsize_t theta);
```

- 函数参数:

theta: 单精度浮点输入参数

- 函数返回值:

theta 的余弦值。

- 函数性能:

函数名称	数据大小	周期数
cosf	Test Size: 512	89

3.4. divf

- 功能描述:

单精度浮点除法运算。

- 函数原型:

```
fsize_t divf(const fsize_t num, const fsize_t den);
```

- 函数参数:

num: 运算的分子;

den: 运算的分母。

- 函数返回值:

返回 num/den 的计算结果。

- 函数性能:

函数名称	数据大小	周期数
divf	Test Size: 512	64

3.5. expf

- 功能描述:

单精度浮点数的指数运算。

- 函数原型:

```
fsize_t expf(const fsize_t x);
```

- 函数参数:

x: 单精度浮点输入。

- 函数返回值:

计算结果。

- 函数性能:

函数名称	数据大小	周期数
expf	Test Size: 512	173

- 注意点

输入域限制为 $\pm \log(\text{FLT_MAX})$ ($< \pm 89$)

3.6. isqrtf

- 功能描述:

单精度浮点数下的平方根倒数运算。

- 函数原型:

```
fsize_t isqrtf(const fsize_t x);
```

- 函数参数:

x: 单精度浮点输入。

- 函数返回值:

返回浮点参数 X 的平方根的倒数。

- 函数性能:

函数名称	数据大小	周期数
isqrtf	Test Size: 512	48

3.7. logf

- 功能描述:

单精度浮点数的自然对数运算。

- 函数原型:

```
float logf(const float x);
```

- 函数参数:

x: 单精度浮点输入。

- 函数返回值:

返回浮点数参数 X 的自然对数值。

- 函数性能:

函数名称	数据大小	周期数
logf	Test Size: 512	96

3.8. powf

- 功能描述:

单精度浮点数的幂函数运算。

- 函数原型:

```
float powf(const float x, const float y);
```

- 函数参数:

x: 运算底数;

y: 运算指数;

- 函数返回值:

范围 x^y 的计算结果

- 函数性能:

函数名称	数据大小	周期数
powf	Test Size: 128	199

- 注意点

输入 x 必须大于或等于 1

3.9. **sinf**

- 功能描述:

单精度浮点数下的正弦函数（以弧度为单位）。

- 函数原型:

```
fsize_t sinf(const fsize_t theta);
```

- 函数参数:

theta: 单精度浮点输入。

- 函数返回值:

theta 的正弦值。

- 函数性能:

函数名称	数据大小	周期数
sinf	Test Size: 512	83

3.10. **sincosf**

- 功能描述:

单精度浮点数下的正弦和余弦函数（以弧度为单位）。

- 函数原型:

```
sincos_t sincosf(const fsize_t theta);
```

- 函数参数:

theta: 单精度浮点输入。

- 函数返回值:

theta 的正弦和余弦值。sincos_t 的定义如下:

```
typedef struct{
    fsize_t c;    // The Cosine
    fsize_t s;    // The Sine
}sincos_t;
```

- 函数性能:

函数名称	数据大小	周期数
sincosf	Test Size: 128	129

3.11. sqrtf

- 功能描述:

单精度浮点平方根运算。

- 函数原型:

```
fsize_t sqrtf(const fsize_t x);
```

- 函数参数:

x: 单精度浮点输入。

- 函数返回值:

x 的平方根。

- 函数性能:

函数名称	数据大小	周期数
sqrtf	Test Size: 512	39